

Machine Learning LAB

Week 03

Lab Contents

Logistic Regression

Polynomial Regression

Logistic Regression (Scratch)

- Logistic Regression from scratch

Logistic Regression (Scratch)

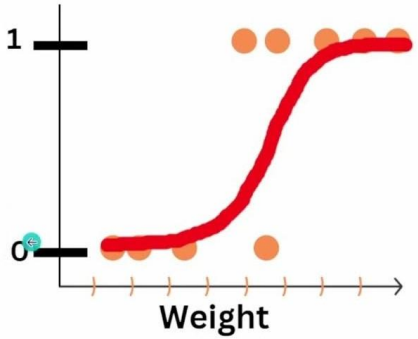
- it's also known as logit regression
- Use for classification.

- Help in decision making.
- Where the situation is not clear you do not know the outcome results.
- in regression dependent variable was continuous.
- In this dependent variables is binary 0 or 1 or binomial.
- Used in decision making so it support probability.
- probability range is 0-1.
- Binary classification it means that only two outcomes.
- Multi class means different classes.

Logistic Regression (Scratch)

- in linear relation we can draw straight line to classify but in this we can't.
- The outcome is form of s.

- Backend working sigmoid or logit function
- Sigmoid probability is equal to 1 / 1 + e^{-z}. So e Euler's number = 2.7
- Z is equal to linear equation equation y= mx+b...
- it converts input into 0 1 range...



Logistic Regression (Scratch)

- import numpy as np
- import pandas as pd
- from sklearn.model_selection import train_test_split
- from sklearn.preprocessing import StandardScaler, LabelEncoder
- from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
- # Load the dataset from a CSV file
- data = pd.read_csv('/content/Social_Network_Ads.csv')

Logistic Regression (Scratch)

- # Preprocessing
- # Encode categorical variable 'Gender'
- data = pd.get_dummies(data, columns=['Gender'], drop_first=True)
- # Define features and target variable
- X = data[['Gender_Male', 'Age', 'EstimatedSalary']].values
- y = data['Purchased'].values # Target variable

	Gender	Age	Salary
0	Male	22	20000
1	Female	35	50000
2	Male	29	60000
3	Female	40	80000

	Age	Salary	Gender_Female	Gender_Male
0	22	20000	0	1
1	35	50000	1	0
2	29	60000	0	1
3	40	80000	1	0

	Age	Salary	Gender_Male	
0	22	20000	1	# Male
1	35	50000	0	# Female
2	29	60000	1	# Male
3	40	80000	0	# Female

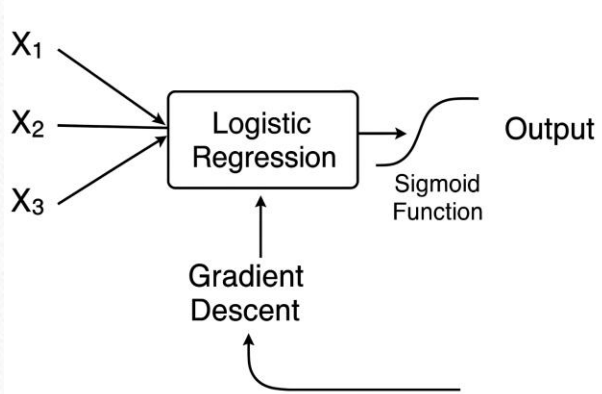
Logistic Regression (Scratch)

- # Split the dataset into training and testing sets
- X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
-
  # Scale the features
- scaler = StandardScaler()
- X_train = scaler.fit_transform(X_train)
- X_test = scaler.transform(X_test)
-

Logistic Regression (Scratch)

```
# Logistic Regression Implementation
class LogisticRegression:
    def __init__(self, learning_rate=0.01, num_iterations=1000):
        self.lr = learning_rate
        self.iterations = num_iterations
        self.weights = None
        self.bias = None
    def sigmoid(self, z):
        return 1 / (1 + np.exp(-z))

    def fit(self, X, y):
        num_samples, num_features = X.shape
        self.weights = np.zeros(num_features)
        self.bias = 0
```



Logistic Regression (Scratch)

```
for i in range(self.iterations):
        linear_model = np.dot(X, self.weights) + self.bias
        y_predicted = self.sigmoid(linear_model)

        # Update weights and bias by calculating Gradient
        dw = (1 / num_samples) * np.dot(X.T, (y_predicted - y))
        db = (1 / num_samples) * np.sum(y_predicted - y)
        self.weights -= self.lr * dw
        self.bias -= self.lr * db

    def predict(self, X):
        linear_model = np.dot(X, self.weights) + self.bias
        return [1 if i > 0.5 else 0 for i in self.sigmoid(linear_model)]
```

$$dw = \frac{1}{n} \cdot X^T \cdot (y_{\text{predicted}} - y)$$

$$db = \frac{1}{n} \cdot \sum (y_{\text{predicted}} - y)$$

Logistic Regression (Scratch)

```
# Train the Model
model = LogisticRegression(learning_rate=0.1, num_iterations=1000)
model.fit(X_train, y_train)

# Evaluate the Model
predictions = model.predict(X_test)
print("Accuracy:", accuracy_score(y_test, predictions))
print("Confusion Matrix:\n", confusion_matrix(y_test, predictions))
print("Classification Report:\n", classification_report(y_test, predictions))
```

```
Accuracy: 0.8875
Confusion Matrix:
 [[50  2]
 [ 7 21]]
Classification Report:
              precision    recall  f1-score   support

           0       0.88      0.96      0.92        52
           1       0.91      0.75      0.82        28

    accuracy                           0.89        80
   macro avg       0.90      0.86      0.87        80
weighted avg       0.89      0.89      0.88        80
```

Logistic Regression (Sklearn)

- import pandas as pd
- from sklearn.model_selection import train_test_split
- from sklearn.preprocessing import StandardScaler, LabelEncoder
- from sklearn.linear_model import LogisticRegression
- from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

- # Load dataset
- data = pd.read_csv("/content/Social_Network_Ads.csv")
- # Encode Gender (Male/Female → 1/0)
- le = LabelEncoder()
- data['Gender'] = le.fit_transform(data['Gender'])

Logistic Regression (Sklearn)

- # Features & target
- X = data[['Gender', 'Age', 'EstimatedSalary']].values
- y = data['Purchased'].values

- # Train-test split
- X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

- # Scale features
- scaler = StandardScaler()
- X_train = scaler.fit_transform(X_train)
- X_test = scaler.transform(X_test)

Logistic Regression (Sklearn)

- # Logistic Regression
- model = LogisticRegression()
- model.fit(X_train, y_train)
- # Predictions
- y_pred = model.predict(X_test)

```
Accuracy: 0.8875
Confusion Matrix:
 [[50  2]
 [ 7 21]]
Classification Report:
              precision    recall  f1-score   support

           0       0.88      0.96      0.92        52
           1       0.91      0.75      0.82        28

    accuracy                           0.89        80
   macro avg       0.90      0.86      0.87        80
weighted avg       0.89      0.89      0.88        80
```

- # Evaluation
- print("Accuracy:", accuracy_score(y_test, y_pred))
- print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred))
- print("Classification Report:\n", classification_report(y_test, y_pred))

Polynomial Regression

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score

# Load dataset
data = pd.read_csv("Position_Salaries.csv")

# Independent variable (Level) and dependent variable (Salary)
X = data[["Level"]].values
y = data["Salary"].values
```

Polynomial Regression

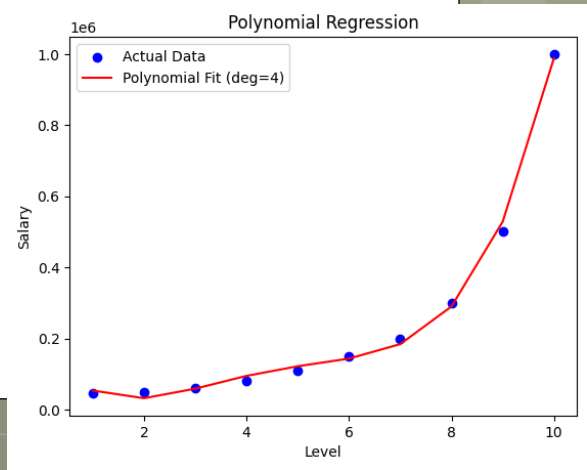
- # Polynomial transformation (degree=4 for better curve fitting)
- poly = PolynomialFeatures(degree=4)
- X_poly = poly.fit_transform(X)

- # Train polynomial regression model
- poly_model = LinearRegression()
- poly_model.fit(X_poly, y)

- # Predictions
- y_pred = poly_model.predict(X_poly)

Draw the line….

- # Plot results
- plt.scatter(X, y, color="blue", label="Actual Data")
- plt.plot(X, y_pred, color="red", label="Polynomial Fit (deg=4)")
- plt.xlabel("Level")
- plt.ylabel("Salary")
- plt.title("Polynomial Regression")
- plt.legend()
- plt.show()



Draw the line….

- # Calculate R2 score
- r2 = r2_score(y, y_pred)
- # Display results
- print("POLYNOMIAL REGRESSION METRIC:")
- print(f"R^2 Score: {r2:.4f}")

```
POLYNOMIAL REGRESSION METRIC:
R² Score: 0.9974
```

Activity 1

Predict whether a customer will **churn** (leave the company) based on their behavior and usage patterns.

Dataset: Telco Customer Churn

Instructions:

1.Load dataset and preprocess categorical variables (Encoding).

2.Train a **Logistic Regression model** with features like `tenure`, `MonthlyCharges`, `Contract`, `InternetService`.

3.Evaluate using **Accuracy, Precision, Recall, F1-score**.

4.Plot **Confusion Matrix & ROC Curve**.

Activity 2

Predict **used car prices** based on mileage, age, and horsepower.
Dataset: Car Price Prediction
Instructions:
1.Choose features like `Kms_Driven`, `Year`, `Engine`, `Horsepower`.
2.Apply **Linear Regression** and record performance.
3.Apply **Polynomial Regression (degree=2,3,4)** and compare results.
4.Plot **fitted curves** for different polynomial degrees.